PÓS-TECH DEVOPS · TECH CHALLENGE FASE 2

Roteiro do Vídeo de Entrega

Projeto: ToggleMaster em microsserviços no AWS EKS · **Duração alvo:** 18 a 19 min (limite de 20) · **Cássio Murilo dos Santos**

Projeto: ToggleMaster em microsserviços no AWS EKS **Duração alvo:** 18 a 19 minutos (o limite do enunciado é 20)

As falas abaixo são um ponto de partida. Fale com as suas palavras — o que importa é cobrir os pontos, não decorar o texto. Os 11 itens que o enunciado exige do vídeo estão todos marcados no roteiro com o selo `EXIGIDO`.

Bloco 0 — Preparação (ANTES de ligar a gravação)

O provisionamento leva uns 25 minutos e **não cabe** nos 20 minutos de vídeo. Faça tudo isto com a gravação desligada, na mesma sessão do laboratório.

0.1 Limpar as sobras da sessão anterior

A limpeza do dia 16/10 foi interrompida quando o token do lab expirou. Pode ter ficado um cluster EKS de pé consumindo orçamento. **Primeira coisa ao abrir o lab:**

```
aws eks list-clusters
aws rds describe-db-instances --query 'DBInstances[].DBInstanceIdentifier'
aws elasticache describe-cache-clusters --query 'CacheClusters[].CacheClusterId'
```

Se aparecer o cluster `togglemaster` antigo, apague e espere, porque o nome precisa estar livre para o cluster novo:

```
aws eks delete-nodegroup --cluster-name togglemaster --nodegroup-name ng-togglemaster
aws eks wait nodegroup-deleted --cluster-name togglemaster --nodegroup-name ng-togglemaster
aws eks delete-cluster --name togglemaster
aws eks wait cluster-deleted --name togglemaster
```

Confira também o orçamento consumido no painel do Academy antes de começar.

0.2 Provisionar tudo

Siga o `docs/runbook-aws.md` do passo 0 ao 9. Resumo do que tem de estar de pé antes de gravar:

Etapa	O que fica pronto
Runbook 2	5 repositórios no ECR com as imagens `v1` publicadas
Runbook 3	Cluster EKS + node group de 2 nós, **e o hop limit do IMDS em 2**
Runbook 4-7	3 RDS, 1 ElastiCache, 1 tabela DynamoDB, 1 fila SQS
Runbook 8	Metrics Server e Nginx Ingress Controller em `1/1`
Runbook 9	Manifestos aplicados, Job do schema concluído, smoke test 5/5

> **Não esqueça o hop limit do IMDS.** É o erro que mais custou tempo na primeira execução: o node group nasce com hop limit 1 e os pods não conseguem credencial da AWS. Está documentado no passo 3 do runbook.

Ao fim do smoke test, cole a `SERVICE_API_KEY` que ele imprime no `k8s/02-secret.yaml`, aplique de novo e reinicie o `evaluation-service`. Guarde essas duas variáveis à mão para a gravação:

```
export INGRESS_HOST=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')
export API_KEY='<a chave que o smoke test imprimiu>'
export QUEUE_URL=$(aws sqs get-queue-url --queue-name togglemaster-events --output text)
echo "http://$INGRESS_HOST"
```

0.3 Subir o ambiente local

O primeiro item do vídeo é o `docker compose`. Suba antes para não gravar os minutos de build:

```
docker compose up -d
docker compose ps        # tem de listar 9 contêineres de pé
```

O `dynamodb-init` aparece como `Exited (0)` — é um job que cria a tabela e encerra, está correto.

0.4 Deixar as janelas abertas

Janela	Conteúdo
Terminal 1	onde você digita os comandos (o protagonista)
Terminal 2	`kubectl -n togglemaster get hpa -w` já rodando
Terminal 3	para ligar a carga e rodar o script do SQS
Navegador aba 1	Console AWS → EKS → cluster `togglemaster`
Navegador aba 2	Console AWS → RDS → lista de instâncias
Navegador aba 3	Console AWS → DynamoDB → `ToggleMasterAnalytics` → Explore items
Navegador aba 4	Console AWS → SQS → `togglemaster-events`
Navegador aba 5	o diagrama da arquitetura
Editor	VS Code aberto na pasta `k8s/`

0.5 Teste de gravação

Grave 30 segundos, assista, confirme que o áudio está bom e que a fonte do terminal está grande o suficiente para ler no vídeo.

AGORA SIM: ligue a gravação

---

1. Abertura — 0:00 a 1:00

O que mostrar: seu rosto, ou um slide com o título.

> Oi, pessoal. Meu nome é Cássio Murilo dos Santos, sou aluno da pós-graduação em DevOps da Pós-Tech, e esse vídeo é a entrega da Fase 2 do Tech Challenge.
>
> Na Fase 1 eu peguei o ToggleMaster, que é uma API de feature flags, e coloquei no ar como um monólito numa instância EC2. Agora na Fase 2 a história é outra: a aplicação foi quebrada em cinco microsserviços, e meu trabalho foi containerizar cada um deles, provisionar a infraestrutura na AWS e colocar tudo rodando num cluster Kubernetes, o EKS, com escalabilidade automática.
>
> Vou mostrar primeiro a arquitetura, depois o ambiente local rodando no Docker Compose, aí a infraestrutura de verdade na AWS, a aplicação funcionando pelo Ingress, e no final a parte que eu achei mais legal: os pods se multiplicando sozinhos quando a carga aumenta. No fechamento eu conto os perrengues.

---

2. Arquitetura e os três bancos — 1:00 a 3:30

EXIGIDO *"Explique a arquitetura implementada e os desafios."* EXIGIDO *"Explique a diferença de propósito entre RDS, ElastiCache e DynamoDB."*

O que mostrar: o diagrama da arquitetura.

> Deixa eu começar explicando o desenho. São cinco microsserviços, e cada um tem uma responsabilidade só.
>
> O **auth-service** cuida das chaves de API: ele emite e valida. O **flag-service** é o CRUD das feature flags. O **targeting-service** guarda as regras de segmentação, por exemplo "libera essa funcionalidade para cinquenta por cento dos usuários". O **evaluation-service** é o coração: ele recebe um usuário e uma flag e responde verdadeiro ou falso — e para isso ele conversa com os outros serviços. E o **analytics-service** é diferente dos quatro: ele não tem API pública nenhuma, é um trabalhador de fila, que fica escutando o SQS e gravando os eventos.
>
> Agora, uma coisa que eu quero destacar, porque foi uma decisão de projeto: esse sistema usa **três tipos diferentes de banco de dados**, e não é exagero, cada um resolve um problema que o outro não resolveria bem.
>
> O **RDS**, que é PostgreSQL, é banco relacional e guarda a verdade do sistema: as chaves, as flags, as regras. É dado que precisa de consistência, de transação, de relacionamento entre tabelas. E repara que eu não usei um RDS só: são **três instâncias separadas**, uma por serviço. Isso é de propósito, é o princípio de banco por serviço — se os cinco compartilhassem uma base, eles não seriam de verdade independentes, e uma mudança de tabela quebraria todos de uma vez.
>
> O **ElastiCache**, que é Redis, é cache em memória. A avaliação de flag é a rota mais chamada do sistema inteiro, e seria um desperdício ir ao banco toda vez para uma resposta que é quase sempre a mesma. O Redis responde em microssegundos e tira essa carga do Postgres. É dado descartável: se o cache cair, o sistema continua funcionando, só fica mais lento.
>
> E o **DynamoDB**, que é NoSQL, guarda os eventos de analytics. Aqui o perfil é completamente diferente: é muita escrita, sem relacionamento, e o volume cresce sem parar. Um banco relacional sofreria com isso. O DynamoDB escala escrita horizontalmente e eu paguei por requisição, no modo on-demand.
>
> Fechando o fluxo: quando chega uma avaliação no evaluation-service, ele consulta o cache, se precisar consulta os outros serviços, responde para o cliente, e **publica um evento no SQS**. A resposta não espera a gravação do analytics — é assíncrono. Lá na frente o analytics-service consome a fila no seu próprio ritmo e grava no DynamoDB. Isso desacopla o caminho rápido do caminho de escrita pesada, e é justamente o que vai me permitir escalar os dois de forma independente.

---

3. Conteinerização e o ambiente local — 3:30 a 5:30

EXIGIDO *"Mostre o `docker compose up` com os 9 contêineres rodando."*

O que mostrar: um Dockerfile no editor, depois o terminal.

Antes de ir para a nuvem, a conteinerização. Cada um dos cinco serviços tem o seu próprio Dockerfile, e todos usam **multi-stage build**. Deixa eu abrir um aqui para explicar o que isso significa.

A ideia é que a construção tem dois estágios. No primeiro eu uso uma imagem cheia de ferramentas, com o compilador, as bibliotecas de desenvolvimento, tudo. No segundo estágio eu começo de uma imagem mínima e copio só o resultado pronto. O compilador não vai para a imagem final.

E a diferença é brutal: os dois serviços escritos em Go ficaram com imagens de **sete megabytes**. Sete. Os de Python ficaram em torno de sessenta e cinco, porque o interpretador tem que ir junto. Imagem pequena sobe mais rápido, gasta menos rede e tem menos coisa instalada para dar problema de segurança.

Agora o ambiente local. Eu tenho um Docker Compose na raiz que sobe o sistema inteiro na minha máquina, sem AWS nenhuma.

```
docker compose ps
```

Olha aqui: são nove contêineres. Os cinco microsserviços, e mais quatro bancos locais — dois Postgres, um Redis e o DynamoDB Local.

Tem dois contêineres a mais aí que eu preciso explicar, porque não estavam na lista do enunciado e foram escolha minha. Esse **elasticmq** é um emulador de SQS: a AWS não tem fila para rodar local, e sem ele o fluxo assíncrono simplesmente não funcionaria na minha máquina. E esse **dynamodb-init** é um serviço de vida curta, que só cria a tabela no DynamoDB Local e encerra — é por isso que ele aparece como "Exited", não é um contêiner com problema.

O detalhe que eu acho mais bonito disso: do compose para o Kubernetes eu **não mudei uma linha de código da aplicação**. O que muda é só a configuração. Local, a variável do SQS aponta para o ElasticMQ; na AWS, ela fica vazia, e aí o SDK usa o endpoint real da Amazon.

---

4. A infraestrutura na AWS — 5:30 a 8:00

`EXIGIDO` *"Mostre o cluster Kubernetes provisionado na nuvem."* `EXIGIDO` *"Explique os desafios, por exemplo as limitações da LabRole."*

O que mostrar: console da AWS, passando pelas abas já abertas.

Vamos para a nuvem de verdade. Aqui é o console da AWS.

Esse é o **cluster EKS**, chamado togglemaster, status Active. O EKS é o Kubernetes gerenciado da Amazon: ela cuida do painel de controle e eu cuido dos nós. E aqui está o **node group**, que são as máquinas onde os meus contêineres rodam de fato: duas instâncias t3.medium, com escala automática configurada de mínimo dois até máximo quatro.

Aqui nos **repositórios do ECR** estão as cinco imagens que eu construí e publiquei, uma por serviço. O EKS puxa as imagens daqui.

No **RDS**, as três instâncias PostgreSQL, uma para cada serviço que precisa de banco relacional, todas privadas, sem acesso pela internet. Esse aqui é o **ElastiCache** com o Redis. Essa é a tabela do **DynamoDB**. E essa é a fila **SQS**, do tipo Standard.

Agora, uma limitação importante do ambiente e que mudou decisões minhas de arquitetura. Eu estou no AWS Academy, que é o laboratório do curso, e nele eu **não tenho permissão de criar IAM Roles**. Tem uma role pronta, a **LabRole**, e é obrigatório usar ela — foi o que eu fiz, tanto no cluster quanto no node group.

Isso parece um detalhe burocrático, mas tem consequência prática em cascata. Sem poder criar role, eu não tenho o IRSA, que é o mecanismo que dá uma identidade da AWS para cada pod individualmente. E sem IRSA, duas ferramentas que seriam a escolha natural ficam fora: o **KEDA**, que escalaria o analytics olhando direto o tamanho da fila, e o **Karpenter**, que escalaria os nós. Eu volto nisso quando falar de escalabilidade.

Ah, e tem uma decisão que eu quero declarar abertamente: eu criei o cluster pela **AWS CLI**, com o comando `aws eks create-cluster`, e não clicando no console. O enunciado pedia o console e proibia o `eksctl` — e o `eksctl` eu não usei, justamente porque ele tenta criar IAM Roles e aqui isso não é permitido. A CLI é provisionamento manual igual ao console, comando por comando, com a LabRole como exigido, e com a vantagem de ficar registrado e reproduzível no runbook que eu escrevi.

---

5. O cluster por dentro: os manifestos — 8:00 a 11:00

`EXIGIDO` *"Mostre os 5 microsserviços rodando como Pods."* `EXIGIDO` (Requisito 3) Metrics Server e Nginx Ingress Controller instalados.

O que mostrar: terminal, e o VS Code na pasta `k8s/`.

```
kubectl get nodes
kubectl -n togglemaster get pods
```

Agora vamos por dentro do cluster. Esses são os dois nós, prontos. E aqui estão os pods: os cinco microsserviços rodando. Repara que os quatro primeiros têm duas réplicas cada, e o analytics tem uma só — ele vai crescer sozinho depois, quando eu der carga.

Reparem também que tudo está dentro de um **namespace** chamado togglemaster. Namespace é como uma pasta lógica dentro do cluster: isola os meus recursos dos add-ons da infraestrutura, que ficam em outros namespaces.

```
kubectl -n togglemaster get svc,ingress,hpa
```

Aqui estão os **Services**, todos do tipo ClusterIP. ClusterIP significa que eles só existem dentro do cluster, não são expostos para fora. E eles fazem duas coisas: dão um nome de DNS fixo e distribuem a carga entre as réplicas. Isso é o que faz os serviços se acharem — quando o evaluation chama `http://auth-service:8001`, quem resolve esse nome é o DNS do cluster, e ele entrega para qualquer um dos pods que estiver saudável.

> E esse é o **Ingress**, que é a única porta de entrada pública, com o endereço do load balancer. Já volto nele.

Agora abra a pasta `k8s/` no editor e passe pelos arquivos.

> Deixa eu mostrar os manifestos, porque é aí que está o trabalho.
>
> Esse é o **ConfigMap**, que é onde fica a configuração que não é secreta: a região da AWS, o nome da tabela, a URL da fila, e as URLs internas dos serviços.
>
> E esse é o **Secret**, que guarda o que é sensível: as senhas e os endereços dos bancos, e as chaves da aplicação. Deixa eu mostrar como ele fica dentro do cluster:

```
kubectl -n togglemaster get secret togglemaster-secrets -o yaml
```

> Olha isso: os valores aparecem nesse formato embaralhado, que é **base64** — era o que o enunciado pedia. Mas aqui eu quero ser honesto sobre uma coisa que eu aprendi na aula: **base64 não é criptografia**. É só codificação. Qualquer pessoa decodifica isso em um comando, não tem senha nenhuma protegendo. O que o Secret resolve de verdade é não deixar a senha espalhada pelos manifestos e no histórico do Git, e permitir controlar quem tem acesso a ele. Proteção de verdade seria criptografia no etcd ou o AWS Secrets Manager, que eu não usei aqui porque está fora do escopo da fase.
>
> Agora, dentro do manifesto de cada serviço, duas coisas que eu quero mostrar.
>
> A primeira são os **requests e limits**. O request é quanto o pod reserva: é com esse número que o Kubernetes decide em qual nó ele cabe. O limit é o teto que ele não pode passar. Se faltar request, o Kubernetes empilha pods demais num nó e tudo fica lento; se faltar limit, um pod com problema come a máquina inteira e derruba os vizinhos. Eu coloquei nos cinco serviços.
>
> A segunda são as **probes**, e aqui tem uma diferença que eu achei a parte mais interessante de aprender. Eu uso três tipos. A **startupProbe** dá um tempo para o pod iniciar sem ser julgado. A **livenessProbe** pergunta "você está vivo?" — se falhar, o Kubernetes mata e reinicia o pod. E a **readinessProbe** pergunta "você está pronto para receber tráfego?" — se falhar, ele só tira o pod do balanceamento, sem reiniciar.
>
> E por que isso importa? Porque a liveness aponta para uma rota que **não consulta o banco**, e a readiness aponta para uma que consulta. Se eu tivesse colocado o banco na liveness, um soluço no RDS reiniciaria os pods de todos os serviços ao mesmo tempo, em cascata, e aí eu transformaria um problema pequeno numa queda geral. Com a separação, se o banco oscila, os pods saem do balanceamento, esperam, e voltam quando o banco voltar. Sem reinício nenhum.

```
kubectl -n kube-system get deploy metrics-server
kubectl -n ingress-nginx get pods
```

> E esses dois são os add-ons que eu instalei no cluster. O **Metrics Server** coleta o consumo de CPU e memória dos pods — sem ele o autoescalamento simplesmente não funciona, ele não tem como saber se está na hora de crescer. E o **Nginx Ingress Controller**, que é quem transforma o meu manifesto de Ingress em roteamento de verdade.

6. A aplicação funcionando pelo Ingress — 11:00 a 12:30

EXIGIDO *"Mostre o Nginx Ingress funcionando (curl para a URL do Load Balancer)."*

O que mostrar: terminal, e o arquivo `20-ingress.yaml` no editor.

> Agora vamos provar que funciona de fora. Esse é o manifesto do Ingress, e aqui estão as **regras de roteamento**: tudo que chegar em barra evaluate vai para o evaluation-service na porta 8004; barra flags vai para o flag-service; barra rules para o targeting; e barra validate e barra admin para o auth.
>
> Repara que o **analytics não tem rota nenhuma** aqui, e isso é de propósito: ele é trabalhador de fila, não tem API pública. Ninguém chama o analytics de fora, ele vai buscar trabalho no SQS.
>
> Quando eu apliquei esse manifesto, o controller do Nginx provisionou um **Network Load Balancer** de verdade na AWS. É esse endereço aqui.

```
echo "http://$INGRESS_HOST"
curl -i "http://$INGRESS_HOST/flags"
```

> Primeiro eu chamo sem credencial nenhuma... e olha: **401, não autorizado**. Ótimo, isso é o comportamento certo — a API está protegida, e o mais importante, isso prova que o tráfego atravessou o load balancer, entrou no Nginx, foi roteado para o flag-service, e o flag-service consultou o auth-service para validar. Toda a cadeia funcionou nessa resposta de erro.

```
curl -s -H "Authorization: Bearer $API_KEY" "http://$INGRESS_HOST/flags"
```

> Agora com a chave de API... e aí está a flag que eu criei, com a descrição e o status. Essa chave, por sinal, foi emitida pelo próprio auth-service rodando no cluster.

```
curl -s "http://$INGRESS_HOST/evaluate?user_id=video-demo&flag_name=enable-new-dashboard"
```

> E essa é a rota principal do sistema: eu pergunto se o usuário "video-demo" deve ver a nova funcionalidade... e ele respondeu. Essa única chamada passou pelo load balancer, pelo Nginx, pelo evaluation-service, que consultou o Redis, o flag-service e o targeting-service, e ainda publicou um evento na fila do SQS. Tudo isso nesse milissegundos aí.

7. Escalabilidade: o evaluation-service sob carga — 12:30 a 15:00

EXIGIDO *"Gere carga no evaluation-service e mostre o HPA criando novos Pods."*

O que mostrar: tela dividida — o terminal do `get hpa -w` e o da carga.

> Agora a parte da escalabilidade automática, que é o coração da Fase 2.
>
> Quem faz isso é o **HPA**, o Horizontal Pod Autoscaler. Ele funciona assim: eu digo qual o alvo de uso de CPU, e ele compara o consumo real com o que o pod reservou. Eu configurei o alvo em **sessenta por cento**. Quando a média dos pods passa disso, ele cria mais réplicas; quando cai, ele reduz.
>
> Nesse terminal eu estou observando os dois HPAs ao vivo. Repara no estado de agora: o evaluation está com duas réplicas, o analytics com uma, e o uso de CPU está baixinho, perto de zero, porque ninguém está usando o sistema.
>
> Vou ligar a carga agora. Eu tenho um gerador de carga preparado como um Deployment, que fica com zero réplicas até eu querer. Cada pod dele dispara vinte chamadas em paralelo, sem parar, no evaluation-service, com usuários aleatórios.

```
kubectl -n togglemaster scale deploy/loadgen --replicas=3
```

> Ligado. Agora é só olhar o HPA.
>
> *(espere de 30 a 60 segundos, narrando)*
>
> Olha ali, a coluna de CPU começou a subir... passou dos sessenta por cento do alvo... e agora o número de réplicas mudou. Ele está criando pods novos.
>
> Isso aqui não é truque de configuração, é o cluster reagindo. O Metrics Server mediu o consumo, o HPA comparou com o alvo, decidiu que precisava de mais gente, e o Deployment criou os pods. Os Services já estão mandando tráfego para os pods novos assim que eles ficam prontos — e quem decide que estão prontos é a readinessProbe que eu mostrei antes.

```
kubectl -n togglemaster get pods -l app=evaluation-service
kubectl -n togglemaster top pods
```

> Aqui está a lista de pods do evaluation. Dá para ver pela idade deles quais são os recém-nascidos. E aqui o consumo real de cada um.
>
> O que eu achei interessante na primeira vez que rodei isso: ele não vai direto no máximo. Ele sobe, mede de novo, e vai parando onde a conta fecha — encontra o equilíbrio em que a média volta para perto do alvo. É um laço de realimentação, não uma regra fixa.

---

8. Escalabilidade em cascata: SQS manual, analytics e DynamoDB — 15:00 a 18:00

`EXIGIDO` *"Envie manualmente várias mensagens para a fila SQS."* `EXIGIDO` *"Mostre o HPA do analytics-service detectando a carga e escalando."* `EXIGIDO` *"Mostre os dados aparecendo na tabela do DynamoDB."* `EXIGIDO` *"Explique e justifique a estratégia de escalabilidade do analytics."*

O que mostrar: terminal do SQS, o `get hpa -w`, o console do SQS e do DynamoDB.

Agora o outro lado da escalabilidade, e aqui eu quero explicar bem a **estratégia**, porque foi uma decisão forçada pelo ambiente.

O jeito ideal de escalar um trabalhador de fila é olhar **o tamanho da fila**: se tem dez mil mensagens esperando, suba mais consumidores. A ferramenta que faz isso no Kubernetes é o **KEDA**. E eu não pude usar o KEDA — porque, como eu expliquei, ele precisa de uma identidade da AWS por pod, o IRSA, e para isso eu teria que criar uma IAM Role, o que o AWS Academy não permite.

Então eu usei o caminho que o próprio enunciado indica, e que funciona bem aqui: escalar o analytics por **CPU**. A lógica é indireta mas sólida: se a fila enche, o trabalhador processa mais mensagens, processar mensagem gasta CPU, a CPU sobe, e o HPA sobe os pods. Com a fila vazia é o contrário — ele fica dormindo esperando mensagem, a CPU cai para perto de zero, e o HPA reduz. A CPU funciona como um termômetro do tamanho da fila.

Agora, no bloco anterior eu enchi a fila **indiretamente**, pela carga no evaluate, porque cada avaliação publica um evento. Mas eu quero provar que a fila e o consumidor são independentes da aplicação. Então vou mandar mensagens **direto para o SQS**, pela linha de comando, sem passar por nenhum microsserviço.

```
kubectl -n togglemaster scale deploy/loadgen --replicas=0   # desliga a carga anterior
bash scripts/sqs-send-manual.sh
```

Esse script monta lotes de dez mensagens e manda para a fila com a AWS CLI, com vários remetentes em paralelo, por noventa segundos. Nada disso toca a aplicação — é a fila recebendo mensagem direto de fora.

E tem um detalhe aí que me deu trabalho e vale contar: o corpo da mensagem tem que ter **exatamente** as quatro informações que o analytics espera — usuário, nome da flag, resultado e horário. Se faltar uma, o analytics não consegue gravar, e como ele só apaga a mensagem da fila **depois** de gravar com sucesso, essa mensagem volta para a fila e fica girando para sempre. Isso tem até nome: *poison pill*, pílula envenenada. Eu usei um nome de flag diferente nessas mensagens, "manual-sqs-demo", justamente para poder provar depois que foram essas que chegaram no banco.

Mostre o console do SQS, aba de monitoramento, e o terminal do HPA.

```
aws sqs get-queue-attributes --queue-url "$QUEUE_URL" \
  --attribute-names ApproximateNumberOfMessages
```

Aqui está a fila com as mensagens acumuladas, esperando. E agora olha o terminal do HPA...

(narre o que a tela mostrar)

O analytics estava com uma réplica só. A CPU dele disparou, porque ele está consumindo esse monte de mensagem, e o HPA está subindo os pods. Essa é a escalabilidade em cascata que eu desenhei: eu não mexi em nada no analytics, ele reagiu sozinho ao trabalho que apareceu na fila.

```
kubectl -n togglemaster logs -l app=analytics-service --tail=15
```

> E aqui os logs do analytics, mostrando ele gravando evento por evento no DynamoDB.

Agora o console do DynamoDB, em *Explore items*.

```
aws dynamodb scan --table-name ToggleMasterAnalytics --select COUNT --query Count


aws dynamodb scan --table-name ToggleMasterAnalytics --select COUNT \
  --filter-expression 'flag_name = :f' \
  --expression-attribute-values '{":f":{"S":"manual-sqs-demo"}}' \
  --query Count
```

> E essa é a prova final do fluxo. Esse é o total de eventos na tabela. E essa segunda contagem filtra só pela flag "manual-sqs-demo" — ou seja, são exatamente as mensagens que eu mandei na mão para o SQS agora, que o analytics consumiu e gravou. O caminho inteiro fechou: fila, trabalhador, banco.
>
> Aqui no console dá para ver os itens de verdade, com o usuário, a flag, o resultado e o horário de cada avaliação.

```
kubectl -n togglemaster get hpa
```

> Uma última coisa sobre o HPA: agora que eu parei a carga, ele vai reduzir os pods — mas de propósito **devagar**. Eu configurei uma janela de espera de dois minutos antes de começar a diminuir. Se ele reduzisse na mesma hora, uma carga que vai e volta ficaria criando e destruindo pods sem parar, e isso tem nome também: *flapping*. É melhor pagar um pouco mais de recurso por dois minutos do que ficar instável.

---

9. Desafios e encerramento — 18:00 a 19:30

O que mostrar: seu rosto, ou a seção de desafios do relatório.

> Para fechar, três perrengues de verdade, porque acho que mostram o trabalho por trás disso.
>
> O primeiro, e o que mais me custou tempo, foi o **IMDS**. Os meus pods pegam credencial da AWS a partir da role da máquina onde eles rodam, e isso é buscado num endereço interno de metadados. O problema é que esse endereço tem um limite de "saltos" de rede, e o node group do EKS nasce com esse limite em um — o que basta para a máquina, mas não para um contêiner rodando dentro dela, que é um salto a mais. O sintoma era confuso: o analytics subia, ficava "rodando", mas nunca ficava pronto, e nos logs aparecia "não foi possível localizar credenciais". A correção foi aumentar esse limite para dois. E teve uma segunda parte: aumentar não resolveu na hora, porque o SDK da AWS guarda a falha em cache. Eu tive que reiniciar os pods. Esse é o tipo de coisa que você só aprende quebrando.
>
> O segundo foi um **erro 502** na rota de avaliação. Olhando o log do evaluation, ele dizia que o flag-service tinha respondido 401. O motivo era bobo e óbvio depois: a chave que o evaluation usa para chamar os outros serviços ainda estava com o valor de exemplo no Secret, porque a chave real só pode ser emitida

> depois que o auth-service está no ar. Eu emiti, atualizei o Secret... e continuou falhando. Aí aprendi outra: **mudar um Secret não reinicia os pods**. Quem já leu a variável de ambiente continua com o valor velho. Tive que reiniciar na mão.
>
> E o terceiro foi o **webhook do Nginx**. Quando eu apliquei todos os manifestos de uma vez, o Ingress falhou. É que o Nginx instala um validador que confere todo objeto de Ingress antes de aceitar — e os pods dele ainda não estavam prontos para validar. A solução foi só ordem: esperar o controller e aplicar o Ingress por último.
>
> Uma surpresa boa para fechar: o RDS me entregou uma versão de PostgreSQL bem mais nova do que a que eu uso no ambiente local, e eu já esperava ter que ajustar algo. Não precisei mexer em nada — nem nos drivers, nem no SQL. Foi o contrário do susto da Fase 1, onde uma versão de Python nova quebrou uma biblioteca do projeto.
>
> Tudo isso, e mais detalhes, está no relatório escrito que acompanha a entrega, junto com o link do repositório. Muito obrigado por assistir.

Depois de parar a gravação

1. **Limpe a infraestrutura**, na ordem certa — o Ingress antes do cluster, ou o load balancer fica órfão. A seção *Limpeza* do runbook tem a sequência.
2. Confirme com `aws eks list-clusters` e `aws rds describe-db-instances` que não sobrou nada, e **confira o erro de cada comando** — foi exatamente aqui que a limpeza da sessão passada falhou em silêncio.
3. Restaure os placeholders dos manifestos antes de commitar: `git checkout -- k8s/`
4. Derrube o ambiente local: `docker compose down`

Checklist dos 11 itens exigidos do vídeo

#	Item do enunciado	Bloco
1	`docker compose up` com os 9 contêineres	3
2	Cluster Kubernetes provisionado na nuvem	4
3	Os 5 microsserviços rodando como Pods	5
4	Nginx Ingress funcionando (curl no load balancer)	6
5	Carga no evaluation-service + HPA criando pods	7
6	Envio **manual** de mensagens para o SQS	8
7	HPA do analytics-service escalando	8
8	Dados aparecendo no DynamoDB	8
9	Arquitetura e desafios (limitações da LabRole)	2, 4 e 9
10	Justificativa da estratégia de escalabilidade do analytics	8
11	Diferença de propósito entre RDS, ElastiCache e DynamoDB	2

Dicas práticas

- Fonte do terminal grande. O avaliador precisa ler.
- Não precisa ser tomada única: errou, para, respira e regrava só o bloco.
- Os blocos 7 e 8 têm espera. Use o tempo para explicar, não fique calado olhando a tela.
- Se um número na sua tela não bater com o que está no roteiro, fale o número da sua tela. Cada execução dá valores diferentes, e isso é normal.
- Fique de olho no relógio: o limite é 20 minutos e o corte costuma ser automático.